

Task 5: OOPs concepts – class, encapsulation

- Concept: OOP - Encapsulation
- Goal: Instead of loose data and method. Create a Class Student with

Data:	Student name, grades
Subject list	static String[] containing your subject names: Math, Science, Social, English, and French
Constructor	In Constructor, ask user to input grades for above subjects.
Private Method	fillGradesArray() Instead of asking the user for a raw total number of classes, the program automatically loops through these specific subjects, prompts for each one by name, and maps them to their respective positions in the array.
Private Method	Instead of formatting in Class Main. Format here. private String formatName(String rawName)
Public Method	calculateAverage() Loop through data: grades[] and calculate grade.

```
// — MAIN RUNNER EXECUTION SCRIPT —
public class UnifiedTracker {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        // --- DATA COLLECTION INTAKE ---
        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name (e.g., ' john doe '): ");
        String rawName = inputScanner.nextLine();

        // --- OBJECT BIRTH SYNTHESIS ---
        // We instantiate the object. It handles text cleaning and sequential
        // subject tracking completely on its own inside the constructor!
        Student activeStudent = new Student(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n=====");
        System.out.println("          ACADEMIC REPORT          ");
        System.out.println("=====");
        System.out.println("Student Profile: " + activeStudent.name);
        System.out.println("Total Classes:   " + activeStudent.grades.length);

        // Call instance method to compute arithmetic output
        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average:   " + computedAverage + "%");
        System.out.println("=====");

        inputScanner.close();
    }
}
```

```
// Save this entire file as: UnifiedTracker.java
import java.util.Scanner;

// — FULLY ENCAPSULATED OBJECT BLUEPRINT —
class Student {
    // Shared structural template for subjects
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    // Attributes
    public String name;
    public int[] grades;

    // — REFACTORED AUTOMATED CONSTRUCTOR —
    public Student(String rawName, Scanner inputScanner) {
        // 1. Call private helper to format text upon birth
        this.name = formatName(rawName);

        // 2. Initialize internal array capacity to match the total number of subjects
        this.grades = new int[SUBJECTS.length];

        // 3. Call private helper to run subject-by-subject loops and error safeguards
        fillGradesArray(inputScanner);
    }

    // — PRIVATE INSTANCE HELPER METHODS —

    // Function 1: Pure internal text processing (Private, instance method)
    private String formatName(String rawName) {
        String cleanName = rawName.trim();
        if (cleanName.isEmpty()) return "";
        return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
    }

    // Function 2: Enumerates through subject strings directly into 'this.grades'
    private void fillGradesArray(Scanner inputScanner) {
        System.out.println("\n--- Entering Grades for Class Schedule ---");
        for (int i = 0; i < this.grades.length; i++) {
            // Prompts specifically by the subject name
            System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
            String textGrade = inputScanner.next();

            try {
                this.grades[i] = Integer.parseInt(textGrade);
            } catch (NumberFormatException e) {
                System.out.println("❌ Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
                this.grades[i] = 0;
            }
        }
    }

    // Function 3: Mathematical engine targeting 'this.grades' automatically
    public double calculateAverage() {
        int sum = 0;
        for (int score : this.grades) {
            sum += score;
        }
        return (double) sum / this.grades.length;
    }
}

```